

Helix Thready — SDK Codegen (buf + openapi-generator)

Table of Contents

1. What this is

2. The two-plane pattern

3. Prerequisites

4. Quick start

5. Targets

6. Per-language matrix

7. Anti-drift & round-trip gates

8. What is real vs. scaffold (anti-bluff)

9. Open items

Rev	Date	Author	Change
1	2026-07-22	swarm (API & SDKs — materials)	Initial Makefile + buf config scaffolds implementing the helix_proto pattern from sdk-strategy.md
2	2026-07-22	swarm (docs export)	Fixed inline mermaid syntax so diagram renders

Table of Contents

1. What this is

2. The two-plane pattern

3. Prerequisites

4. Quick start

5. Targets

6. Per-language matrix

7. Anti-drift & round-trip gates

8. What is real vs. scaffold (anti-bluff)

9. Open items

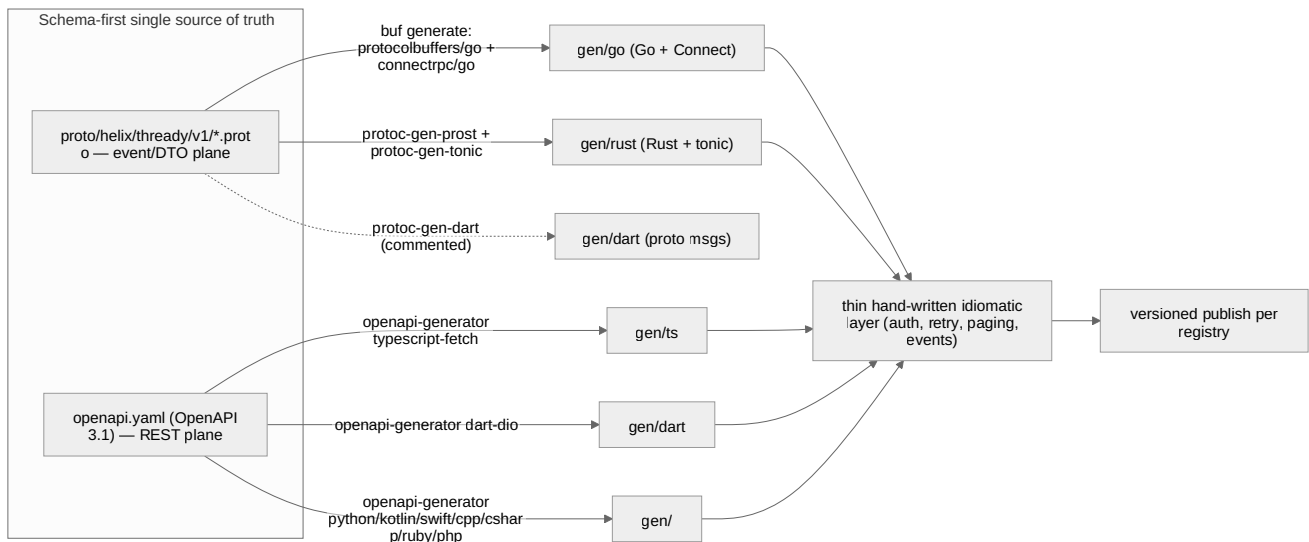
1. What this is

A **runnable codegen harness** that turns the two schema-first contracts into the 11 target SDKs, implementing the `vasic-digital/helix_proto` pattern specified in `sdk-strategy.md` [VERIFIED: `helix_proto buf.gen.yaml/Makefile`] [GAP: #11 SDK generation] . Files here:

File	Purpose
<code>Makefile</code>	All codegen + gate targets (<code>buf</code> proto plane, <code>openapi-generator</code> REST plane, lint, drift, round-trip).
<code>buf.gen.yaml</code>	buf v2 plugin wiring: <code>protocolbuffers/go</code> , <code>connectrpc/go</code> , <code>protoc-gen-prost</code> , <code>protoc-gen-tonic</code> , (<code>dart</code> commented).
<code>buf.yaml</code>	buf module config: <code>modules: [proto]</code> , lint <code>STANDARD</code> , breaking <code>FILE</code> .

The Makefile is deliberately parameterised: by default `OPENAPI` points at this docs area's canonical `../../openapi.yaml` so you can generate REST clients **today** without the `helix_thready_proto` repo. In that repo the same Makefile drives `openapi/thready.v1.yaml` + `proto/helix/thready/v1/*.proto` .

2. The two-plane pattern



Rendered PNG/SVG exported via Docs Chain (§11.4.65).

Explanation (for readers/models that cannot see the diagram). There are two contracts and two generators. The **Protobuf** files (`proto/helix/thready/v1/*.proto`) are the source of truth for the event/DTO wire plane and any streaming RPC; `buf generate` compiles them — via the remote `protocolbuffers/go` and `connectrpc/go` plugins — into Go message types plus Connect service stubs

(`gen/go`), and via the local `protoc-gen-prost` + `protoc-gen-tonic` plugins into a Rust crate with tonic services (`gen/rust`); a `protoc-gen-dart` stanza (kept commented until a Dart proto toolchain is provisioned) would emit Dart proto messages. The **OpenAPI 3.1** document is the source of truth for the synchronous REST surface; `openapi-generator` compiles it into the TypeScript client (`typescript-fetch`), the Dart client (`dart-dio`), and the remaining REST clients (Python, Kotlin/JVM, Swift, C++, C#, Ruby, PHP). Every generated core is then wrapped by a small, hand-written idiomatic layer — auth, retry/back-off, pagination iterators, event-stream helpers — and each wrapped SDK is versioned and published to its language registry. Because both contracts are the single source of truth, regeneration is deterministic and hand-edits to `gen/` are caught by `check-no-handwritten`.

3. Prerequisites

Tool	Needed for	This env [VERIFIED]
<code>buf</code> ≥ 1.7	proto plane (<code>lint</code> , <code>breaking</code> , <code>generate</code>)	present — 1.71.0
<code>@redocly/cli</code>	<code>openapi-lint</code>	present — 2.40.0 (via <code>npx</code>)
<code>openapi-generator</code> v7.x	REST clients	not local ; Makefile falls back to Docker <code>openapitools/openapi-generator-cli:v7.9.0</code> (or set <code>OPENAPI_GEN</code>)
<code>protoc-gen-prost</code> , <code>protoc-gen-tonic</code>	Rust proto plugins	install with <code>cargo install protoc-gen-prost protoc-gen-tonic</code>
<code>cargo</code> , <code>node</code> / <code>tsc</code> , <code>docker</code>	<code>rust-build</code> , <code>ts-check</code> , generation	as required per target

Run `make tools` to print what is detected.

4. Quick start

```
cd docs/public/research/mvp/api/materials/codegen

make tools           # show detected toolchain
make openapi-lint    # redocly lint ../../openapi.yaml (see ../validation.md)
make openapi-generate-ts # -> gen/ts (TypeScript client)
make openapi-generate-all # -> gen/<lang> for all 9 REST languages
make generate        # buf -> gen/{go,rust,dart} (needs proto/, else skips cleanly)
make all             # full local pre-tag gate
```

Override inputs without editing the file:

```
make OPENAPI=/repo/helix_thready_proto/openapi/thready.v1.yaml \
  PROTO_DIR=/repo/helix_thready_proto/proto openapi-generate-all
```

5. Targets

Run `make help` for the generated list. Groups:

- **Proto plane:** `lint`, `breaking`, `generate`, `rust-build`.
- **REST plane:** `openapi-lint`, `openapi-generate-`
`{ts,dart,python,kotlin,swift,cpp,csharp,ruby,php}`, `openapi-generate-all`.
- **Gates:** `ts-check` (`tsc --noEmit`), `check-no-handwritten` (`regenerate + git-diff gen/`),
`roundtrip-test` (drive `gen/ts` against a stub incl. a 401 negative-control).
- **Aggregate:** `all` (the full local pre-tag gate — no server CI, `[CONSTITUTION $11.4.156]`), `clean`.

6. Per-language matrix

Mirrors `sdk-strategy.md` \$4 (final request \$13.1):

Language	Core generator	Make target
Go	<code>buf</code> (protocolbuffers/go + connectrpc/go)	<code>generate</code>
Rust	<code>buf</code> (prost + tonic)	<code>generate</code> , <code>rust-build</code>
TypeScript/JS	<code>openapi-generator typescript-fetch</code>	<code>openapi-generate-ts</code>
Dart/Flutter	<code>openapi-generator dart-dio</code> (+ <code>proto-dart</code>)	<code>openapi-generate-dart</code>
Python	<code>openapi-generator python</code> (asyncio)	<code>openapi-generate-python</code>
Kotlin/JVM (Java/Groovy/Scala)	<code>openapi-generator kotlin</code>	<code>openapi-generate-kotlin</code>
Swift	<code>openapi-generator swift5</code>	<code>openapi-generate-swift</code>
C++	<code>openapi-generator cpp-restsdk</code> (+ protobuf C++)	<code>openapi-generate-cpp</code>
C# (Mono)	<code>openapi-generator csharp</code>	<code>openapi-generate-csharp</code>
Ruby	<code>openapi-generator ruby</code>	<code>openapi-generate-ruby</code>
PHP	<code>openapi-generator php</code>	<code>openapi-generate-php</code>
Zig	hand-written over C ABI / REST	— <code>[OPEN: api-3]</code> no first-class generator

7. Anti-drift & round-trip gates

Reused from helix_proto [VERIFIED], run by local git-hooks:

1. `buf lint` + `buf breaking` (proto hygiene + the breaking-change gate, `versioning.md` §5) and `openapi-lint` (OpenAPI hygiene — see `../validation.md`).
2. `generate` → `check-no-handwritten`: regenerate and assert the `gen/` cores were not hand-edited (ergonomics live only in the thin layer, never in generated files).
3. `rust-build` (`cargo`), `ts-check` (`tsc --noEmit`), and `roundtrip-test` (drive the generated TS client against a real stub server incl. a negative-control 401). RED-first skeletons for the round-trip + drift guard are in `../contract-tests.md` §full-automation.

8. What is real vs. scaffold (anti-bluff)

Per CONVENTIONS §7 — do **not** claim a scaffold works:

- **REAL, runnable now** [VERIFIED]: `make tools`, `make openapi-lint` (redocly 2.40.0 runs against the sibling `openapi.yaml`). The `openapi-generate-*` targets are correct `openapi-generator` invocations and run as soon as the CLI/Docker image is available.
- **SCAFFOLD, not yet wired** [OPEN: api-1]: the **proto plane** (`generate`, `rust-build`, `breaking`). `proto/helix/thready/v1/*.proto` does **not** exist here — those files are produced in the `helix_thready_proto` repo. The proto targets **detect the absence and skip with a clear message** (they do not fabricate output). `buf.gen.yaml` / `buf.yaml` are the verified helix_proto pattern, ready for when `proto/` lands.
- **NOT executed in this environment**: no SDK was actually generated here (no network/Docker pull was attempted); the Makefile is validated only for **parse + dry-run** correctness (`make -n`, `make help`, `make tools`). Treat first real generation as the acceptance test.

9. Open items

- [OPEN: api-1] The `.proto` contract is produced with the code in `helix_thready_proto`; the proto targets here are ready-but-inert until it exists.
- [OPEN: api-3] Zig has no first-class generator — hand-written over C ABI / REST.
- [OPEN: sdk-2] One JVM artifact (Kotlin-first) vs. per-language artifacts — confirm with client teams; `openapi-generate-kotlin` currently emits the single Kotlin artifact.
- [OPEN: cg-1] `check-no-handwritten` diffs `gen/` via `git`; in the real SDK repo, ensure `gen/` is committed so the drift diff is meaningful.

Made with love ♥ by Helix Development.